

Air University



Operating System (Lab)

Year 2026

Ms. Qurat Ul Ain

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: April 03, 2026

Lab # 06

Q 01. Write a C program that creates a pipe for communication between a child process and the parent process. The parent process should read a number from the keyboard and send it to the child process, and the child process must get this number, multiply it by 2, and print its table. Make sure to close the appropriate read and write ends.

Solution:

File Creation: Navigated into the lab6 directory on Ubuntu desktop and used the vi editor to create and start writing a new C source file named Q_01.c.

```
usama@ubuntu:~/Desktop$ cd Lab6
usama@ubuntu:~/Desktop/Lab6$ vi Q_01.c
```

Code: This section of Q_01.c initializes a pipe using pipe(pipefd) and forks a child process. It shows the child closing the write end (pipefd[1]) to prepare for reading an integer from the parent. The logic here completes the task: the child multiplies the received number by 2 and uses a for loop to print its multiplication table, while the parent reads a number from the keyboard and sends it through the pipe.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipefd[2]; // pipefd[0] = read end, pipefd[1] = write end
    pid_t pid;
    int number;

    // Create the pipe
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Fork a child process
    pid = fork();
    if (pid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) { // Child process
        // Close the write end - child only reads
        close(pipefd[1]);

        // Read the number sent by parent
        if (read(pipefd[0], &number, sizeof(number)) == -1) {
            perror("child read");
            exit(EXIT_FAILURE);
        }
    }
}
```

```
// Multiply by 2
int result = number * 2;

// Print the multiplication table of 'result' (from 1 to 10)
printf("Child received %d, multiplied by 2 = %d\n", number, result);
printf("Multiplication table of %d:\n", result);
for (int i = 1; i <= 10; i++) {
    printf("%d x %d = %d\n", result, i, result * i);
}

// Close the read end
close(pipefd[0]);
exit(EXIT_SUCCESS);
}
else { // Parent process
    // Close the read end - parent only writes
    close(pipefd[0]);

    // Read a number from keyboard
    printf("Enter a number: ");
    scanf("%d", &number);

    // Send the number to child via pipe
    if (write(pipefd[1], &number, sizeof(number)) == -1) {
        perror("parent write");
        exit(EXIT_FAILURE);
    }

    // Close the write end
    close(pipefd[1]);

    // Wait for child to finish
    wait(NULL);
}

return 0;
}
```

Compilation & Output: Successfully compiled the code using gcc, ran the ./Q_01 executable, entered the number 9, and verified that the child correctly received the value, doubled it to 18, and displayed the full multiplication table.

```
usama@ubuntu:~/Desktop/Lab6$ gcc Q_01.c -o Q_01
usama@ubuntu:~/Desktop/Lab6$ ./Q_01
Enter a number: 9
Child received 9, multiplied by 2 = 18
Multiplication table of 18:
18 x 1 = 18
18 x 2 = 36
18 x 3 = 54
18 x 4 = 72
18 x 5 = 90
18 x 6 = 108
18 x 7 = 126
18 x 8 = 144
18 x 9 = 162
18 x 10 = 180
```

Q 02. Write a C program that creates a pipe and forks a child process. The parent process sends a string message to the child process through the pipe, and the child process receives and prints the message.

Solution:

File creation: Navigated into the Lab6 directory on the Ubuntu desktop and used the vi editor to create and start writing a new C source file named Q_02.c.

```
usama@ubuntu:~/Desktop/Lab6$ vi Q_02.c
```

Code: This section of the source code shows the initialization of a pipe using pipe(pipefd) and the creation of a child process with fork(). Inside the child process (pid == 0), the unused write end of the pipe is closed, and read() is called to await a string message from the parent. The next section contains the parent process logic. The parent closes the unused read end, prompts the user to enter a message via fgets(), cleans up the newline character, and uses write() to send the string through the pipe to the child.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>

#define BUFFER_SIZE 256

int main() {
    int pipefd[2];
    pid_t pid;
    char message[BUFFER_SIZE];

    // Create the pipe
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Fork a child process
    pid = fork();
    if (pid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) { // Child process
        // Close the write end - child only reads
        close(pipefd[1]);

        // Read the string message from parent
        ssize_t bytes_read = read(pipefd[0], message, BUFFER_SIZE - 1);
        if (bytes_read == -1) {
            perror("child read");
            exit(EXIT_FAILURE);
        }
        message[bytes_read] = '\0'; // Null-terminate the string

        // Print the received message
        printf("Child received: %s\n", message);
    }
}
```

```
// Close the read end
close(pipefd[0]);
exit(EXIT_SUCCESS);
}
else { // Parent process
// Close the read end - parent only writes
close(pipefd[0]);

// Read a string from keyboard
printf("Enter a message to send to child: ");
fgets(message, BUFFER_SIZE, stdin);
// Remove trailing newline if present
size_t len = strlen(message);
if (len > 0 && message[len - 1] == '\n')
    message[len - 1] = '\0';

// Send the string to child via pipe
if (write(pipefd[1], message, strlen(message) + 1) == -1) {
    perror("parent write");
    exit(EXIT_FAILURE);
}

// Close the write end
close(pipefd[1]);

// Wait for child to finish
wait(NULL);
}

return 0;
}
```

Compilation & Output: Successfully compiled the program using gcc and ran the executable ./Q_02. The output demonstrates the parent sending the message "Hello! This is Lab Task 6." and the child process correctly receiving and displaying it.

```
usama@ubuntu:~/Desktop/Lab6$ gcc Q_02.c -o Q_02
usama@ubuntu:~/Desktop/Lab6$ ./Q_02
Enter a message to send to child: Hello! This is Lab Task 6.
Child received: Hello! This is Lab Task 6.
```

- End of Lab Task -